
Imersão Profissional: Projeto de Software

Semana 1 — Organização e fundamentos de arquitetura



Prof. Dr. João Choma

UEM

github.com/JoaoChoma

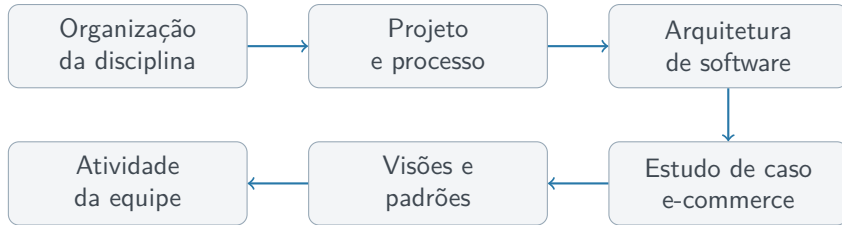
2026/2

Abertura



Ao final desta semana, você deverá ser capaz de:

- compreender a dinâmica da disciplina e a formação dos times;
- reconhecer requisitos e artefatos obrigatórios;
- diferenciar projeto, processo, atividade, tarefa e entregável;
- explicar a importância da arquitetura de software;
- identificar componentes, decisões, visões e padrões;
- aplicar os conceitos a um sistema de e-commerce.



Organização da disciplina



Objetivo geral

Entregar **software de qualidade**, com foco na resolução de problemas reais e na aplicação de boas práticas de Engenharia de Software.

O projeto deve demonstrar:

- responsabilidade técnica e ética;
- colaboração e participação ativa;
- organização e rastreabilidade;
- domínio do que foi desenvolvido;
- integração entre frontend, backend, banco de dados e APIs.



Composição

- Formação livre.
- De **4 a 6 integrantes**.
- Sem trocas após a formação.

Projeto

- Proposta própria ou de parceiro.
- Escopo amplo e tecnicamente adequado.
- Participação de todos nas etapas.

Compromisso

A equipe permanece unida durante o período e responde coletivamente pelo processo e pelo produto.



Backend REST
stateless

Frontend web
HTML5 +
JavaScript

Aplicativo
mobile

Persistência
de dados

Testes
automatizados

Integrações
e APIs

As partes devem formar uma solução integrada, executável e demonstrável.



Recursos externos podem apoiar o desenvolvimento, mas não podem ocultar uma parte relevante da construção do software.

O estudante deve ser capaz de:

- explicar as decisões e o funcionamento do código;
- responder a questionamentos técnicos;
- realizar alterações ao vivo;
- testar e depurar a solução sem aviso prévio.

Princípio

A ferramenta apoia o desenvolvimento; ela não substitui o domínio técnico.



- Casos de uso e classes.
- Diagramas de sequência.
- Diagrama entidade-relacionamento.
- Glossário do negócio.
- Especificações dos casos de uso.
- Critérios de aceitação e testes.
- Mockups das interfaces.
- Documentação da API REST.

Evolução contínua

Os artefatos devem ser atualizados **em paralelo com a implementação.**



Kanban backlog priorizado e fluxo de trabalho visível.

Jira tarefas, responsáveis e registro correto das horas.

Confluence documentação e resultados das cerimônias.

GitHub código e artefatos versionados.

Jenkins integração contínua.

Acompanhamento

Reviews e retrospectivas verificam entregas, processo e próximos passos.



Nota base

- Apresentações internas.
- Apresentações públicas.
- Qualidade do produto.
- Qualidade do processo.

Ajustes individuais

- Avaliação 360°.
- Participação nas tarefas.
- Registros no Jira.
- CRUD incompleto: até 25%.



Nível	Exemplos	Penalidade
Normal	Branches, commits e apontamentos inadequados	−0,5/3
Grave	Faltas, ausência de pushes e tarefas não entregues	−0,5/NC
Gravíssima	Burlar deliberadamente as regras	−1,0/NC
Imperdoável	Ausência em apresentação bimestral	Nota perdida

O processo deixa evidências: tarefas, horas, branches, commits, artefatos e participação nas cerimônias.



- 1 Apresentação da disciplina e do regulamento.
- 2 Revisão de UML: casos de uso e classes.
- 3 Git básico: `init`, `status`, `add`, `commit` e `push`.
- 4 Organização das equipes e definição de papéis.
- 5 Gestão do backlog e criação de sprints no Jira.
- 6 Modelagem do projeto e apresentação dos artefatos.
- 7 Refinamento das entregas.
- 8 Apresentação final dos protótipos.

Projeto e processo



Projeto

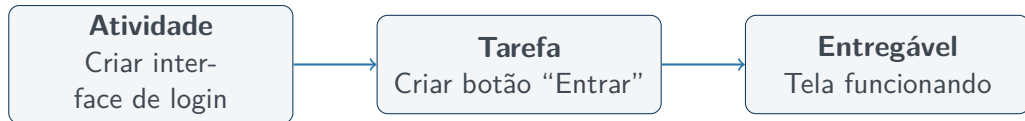
Esforço temporário para criar um produto, serviço ou resultado único.

- Início e fim definidos.
- Objetivo específico.
- Restrições de tempo e recursos.

Processo

Atividades inter-relacionadas que transformam entradas em resultados.

- Contínuo e repetível.
- Estruturado.
- Pode ser aprimorado.



Uma tarefa concluída deve contribuir para um entregável observável e verificável.

Arquitetura de software



Definição

É a estrutura fundamental de um sistema, composta por seus componentes, pelas relações entre eles e pelos princípios que orientam seu projeto e evolução.

A arquitetura abrange:

- concepção e organização inicial;
- implementação;
- manutenção;
- evolução e atualizações.



- Componentes principais do sistema.
- Responsabilidades de cada componente.
- Formas de interação e comunicação.
- Organização em módulos, serviços ou camadas.
- Regras e restrições de desenvolvimento.
- Tecnologias, padrões e interfaces.

Antes dos detalhes

A arquitetura oferece uma estrutura para organizar o sistema antes de detalhar toda a implementação.



Stakeholders

- Desenvolvedores.
- Arquitetos e analistas.
- Gerentes de projeto.
- Scrum Master e PO.
- Clientes.

Uma arquitetura clara:

- cria vocabulário comum;
- alinha objetivos e expectativas;
- explicita capacidades e limitações;
- reduz mal-entendidos;
- permite dividir o trabalho mantendo a visão do todo.



Antes de implementar, a equipe avalia alternativas e toma decisões sobre:

- tecnologias;
- padrões de projeto;
- APIs e integrações;
- requisitos conflitantes;
- desempenho;
- segurança;
- escalabilidade;
- custos.

Trade-off

Toda escolha oferece benefícios, impõe limitações e produz consequências.



Desempenho

Confiabilidade

Usabilidade

Segurança

Escalabilidade

Manutenibilidade

Atributos de qualidade devem orientar as decisões desde o início, não apenas depois da implementação.



Estruturas e componentes módulos, serviços, camadas e interações.

Regras e restrições padrões e diretrizes adotados pela equipe.

Qualidade decisões sobre segurança, desempenho e manutenção.

Decisões arquiteturais escolhas que condicionam adaptação e evolução.



Decisões de projeto afetam:

- a estrutura do código;
- o comportamento e as funcionalidades;
- a capacidade de evolução;
- tecnologias, frameworks e estruturas de dados;
- algoritmos, padrões e divisão de componentes.

Pergunta-chave

Quais decisões serão caras ou arriscadas para mudar depois?



Uma decisão importante deve responder:

- 1 Qual problema precisa ser resolvido?
- 2 Quais alternativas foram consideradas?
- 3 Qual alternativa foi escolhida?
- 4 Por que ela foi escolhida?
- 5 Quais consequências e riscos foram aceitos?

Valor do registro

Preservar o contexto evita que a equipe repita discussões ou trate uma escolha circunstancial como regra absoluta.

Estudo de caso: e-commerce

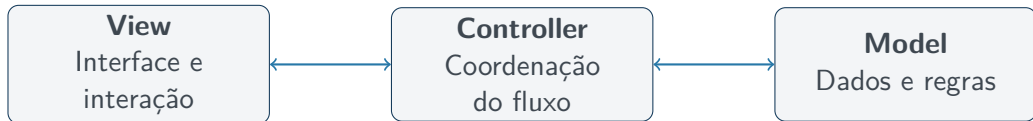


Uma loja vende diferentes produtos pela internet. O sistema deve gerenciar:

- catálogo de produtos;
- clientes e autenticação;
- carrinho e pedidos;
- pagamentos;
- notificações;
- interfaces web e mobile.

Desafio

Como estruturar esses elementos de forma compreensível, testável e evolutiva?



O padrão Model–View–Controller separa apresentação, coordenação e domínio.



Model

Produto, Pedido, Pagamento e Cliente; dados e regras do negócio.

View

Catálogo, carrinho, pagamento e acompanhamento de pedidos.

Controller

Recebe ações, coordena o Model e prepara a resposta.

A separação de responsabilidades favorece organização, teste e manutenção.



Singleton

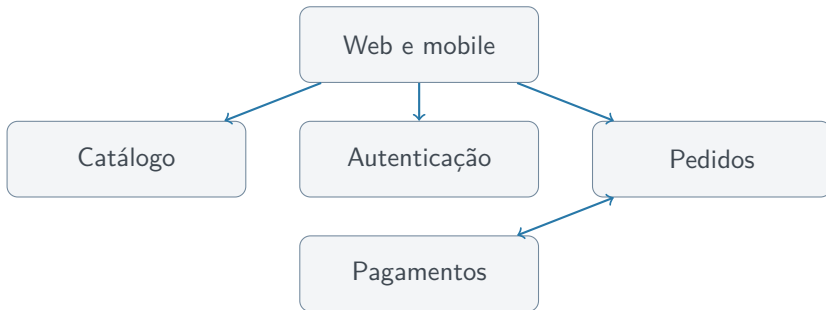
Garante uma única instância de uma classe e um ponto controlado de acesso a um recurso compartilhado.

Factory Method

Cria implementações de pagamento — cartão, boleto ou carteira digital — sem acoplar o fluxo principal às classes concretas.

Contexto

Um padrão deve resolver uma necessidade real; seu uso não é automático.



Essa decomposição é uma alternativa possível, não uma solução universal.



Antes de separar a aplicação em serviços, avalie:

- tamanho e experiência do time;
- necessidade de implantação independente;
- volume e perfil de carga;
- comunicação distribuída e observabilidade;
- complexidade operacional e custo de infraestrutura.

Simplicidade estratégica

Para muitos produtos iniciais, um **monólito modular** é uma opção mais simples e segura.

Visões e padrões



Visões são perspectivas específicas do sistema. Cada uma destaca elementos relevantes para determinado público ou preocupação.

Lógica componentes, serviços e funcionalidade.

Desenvolvimento módulos, pacotes, camadas e código.

Processo execução, concorrência e comunicação.

Física implantação em servidores, dispositivos e infraestrutura.

Por que várias visões?

Uma única representação raramente explica todo o sistema.



Padrões são soluções reutilizáveis para problemas recorrentes de projeto.

- Registram conhecimento consolidado.
- Criam vocabulário comum.
- Oferecem uma estrutura adaptável.
- Não substituem a análise do contexto.

Uso consciente

Um padrão orienta uma solução; ele não deve ser aplicado mecanicamente.



Categoria	Finalidade	Exemplos
Arquitetural	Organizar partes maiores do sistema	MVC, camadas
Criacional	Controlar a criação de objetos	Singleton, Factory
Estrutural	Compor classes e objetos	Adapter, Composite
Comportamental	Distribuir interação e responsabilidades	Observer, Strategy
GRASP	Orientar responsabilidades	Creator, Controller

Atividade da equipe



Em equipe, escolha um problema de software e produza:

- 1 descrição dos usuários e do problema;
- 2 principais funcionalidades;
- 3 componentes iniciais da solução;
- 4 responsabilidades de cada componente;
- 5 uma visão lógica simplificada;
- 6 duas decisões de projeto com justificativas;
- 7 riscos ou atributos de qualidade relevantes.

Objetivo

Tornar as primeiras decisões explícitas e discutíveis, não criar uma arquitetura definitiva.



- ☐ Time formado com 4 a 6 integrantes.
- ☐ Problema e público-alvo definidos.
- ☐ Escopo inicial discutido.
- ☐ Ferramentas de gestão e versionamento preparadas.
- ☐ Backlog inicial criado.
- ☐ Primeira proposta arquitetural registrada.
- ☐ Responsabilidades e próximos passos combinados.

Encerramento



- Projeto é temporário; processo é contínuo e repetível.
- Arquitetura organiza componentes, responsabilidades e relações.
- A arquitetura apoia comunicação e gestão da complexidade.
- Decisões técnicas afetam atributos de qualidade.
- Visões apresentam o sistema sob perspectivas diferentes.
- Padrões ajudam quando aplicados ao contexto correto.
- Artefatos e implementação devem evoluir juntos.



- Que decisões seriam mais difíceis de mudar depois da implementação?
- Quais atributos de qualidade são críticos para o projeto da equipe?
- Como representar a arquitetura para públicos diferentes?
- Quando a separação em serviços realmente se justifica?
- Que evidências demonstram a participação de cada integrante?



Formar a equipe, compreender o problema e registrar as primeiras decisões.

A arquitetura começa quando as escolhas deixam de ser implícitas.